00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 10

Karma (Hashing)

Karma Tabloları ve Desen Eşleme

MODÜL III · DİNAMİK VERİ VE HIZLI ERİŞİM

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** **Karma Tabloları**	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14



TAHMİN ET

Geçen haftadan dört soru (yanıtlar notlarda)

- **S1 (H8).** İki katına büyüyen diziye 10^5 öğe eklenince toplam kaç kopya yapılır?
- **S2 (H9).** `i % k` ile `i & (k - 1)` hangi k için hep aynı? k = 6'da 0 … 999 999 indisin kaçında farklı?
- **S3 (H1).** C'de `-7 % 2` kaç? Python'da?
- **S4 (AP-1 H13).** `strlen("kılıç")` kaç döndürür?

Büyük Soru: aramadan, neredeyse tek adımda bulmak

186 ns

karma tablosu, 10^6 kayıtta numara (ikili arama 762 ns)

107 133

karma yoklaması: 10^5 numarada yinelenen tespiti

4 999 950 000

tüm çiftler: $n(n-1)/2$ karşılaştırma

- Milyonlarca kayıt arasında bir anahtarı aramadan, neredeyse tek adımda nasıl bulursunuz ve bir saldırgan bu “sihri” nasıl bozabilir?

GERÇEK DÜNYADAN

2003

Ortalama O(1)'den O(n)'e: karmaşıklık saldırısı

- Crosby ve Wallach (2003): hepsi aynı kovaya düşen anahtarlarla n ekleme **$\Theta(n^2)$** olur.
- Perl 5.6.1 / 5.8.0: 90 000 kısa saldırı dizgisi hedef sürümde **6506 s** ve **6838 s** işlemci zamanı (öbür sürümde < 2 s).
- Bro saldırı tespit sistemi saniyede **1200** yerine **24** paket işledi.
- Çevirmeli modemden az bant genişliğiyle, altı dakikada trafiğin **%71**'ine varan kısmı düşürüldü.

Crosby, S. A., & Wallach, D. S. (2003). Denial of service via algorithmic complexity attacks. 12th USENIX Security Symposium.



GERÇEK DÜNYADAN

2011 · 2012 · 2013

2011: web platformları ve Python'un yanıtı

- 28C3 (28 Aralık 2011), Klink ve Wälde: “Effective Denial of Service attacks against web application platforms”.
- oCERT-2011-003: Java, JRuby, PHP, Python, Rubinius, Ruby; sorun özellikle POST parametre sayısını sınırlamayan uygulamalarda.
- Python 2.6.8 / 2.7.3 / 3.1.5 / 3.2.3: rastgeleleştirme geldi ama **varsayılan kapalı** (`-R`).
- Python 3.3: **varsayılan açık**; 3.4: `str`/`bytes` karması anahtarlı **SipHash** (PEP 456); 3.11: siphash13.

Klink & Wälde (2011); oCERT (2011); Python Software Foundation (t.y.); Heimes (2013), PEP 456.

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

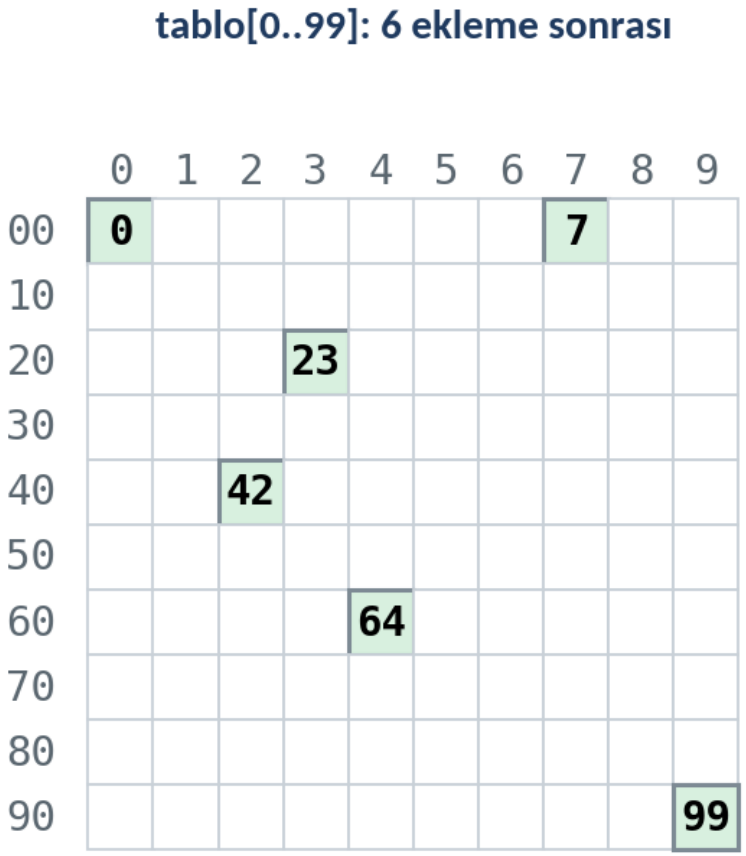
ÖÇ 10.1	Doğrudan adresleme ile karma tablosunu karşılaştırır; iyi bir karma fonksiyonunun özelliklerini açıklar.
ÖÇ 10.2	Ayrı zincirleme ve doğrusal yoklamalı açık adreslemeyle karma tablosu (ekleme, arama, silme) gerçekler.
ÖÇ 10.3	Yük faktörünün başarım üzerindeki etkisini deneyle gösterir; yeniden boyutlandırmayı amortize analizle gerekçelendirir.
ÖÇ 10.4	Karma tablolarını frekans sayma, yinelenen tespiti ve iki toplam gibi problemlerde kullanarak $O(n^2)$ çözümleri ortalama $O(n)$'e indirir.
ÖÇ 10.5	Naif desen eşleme ve Rabin–Karp (kayan karma) algoritmalarını gerçekler ve karmaşıklıklarını karşılaştırır.

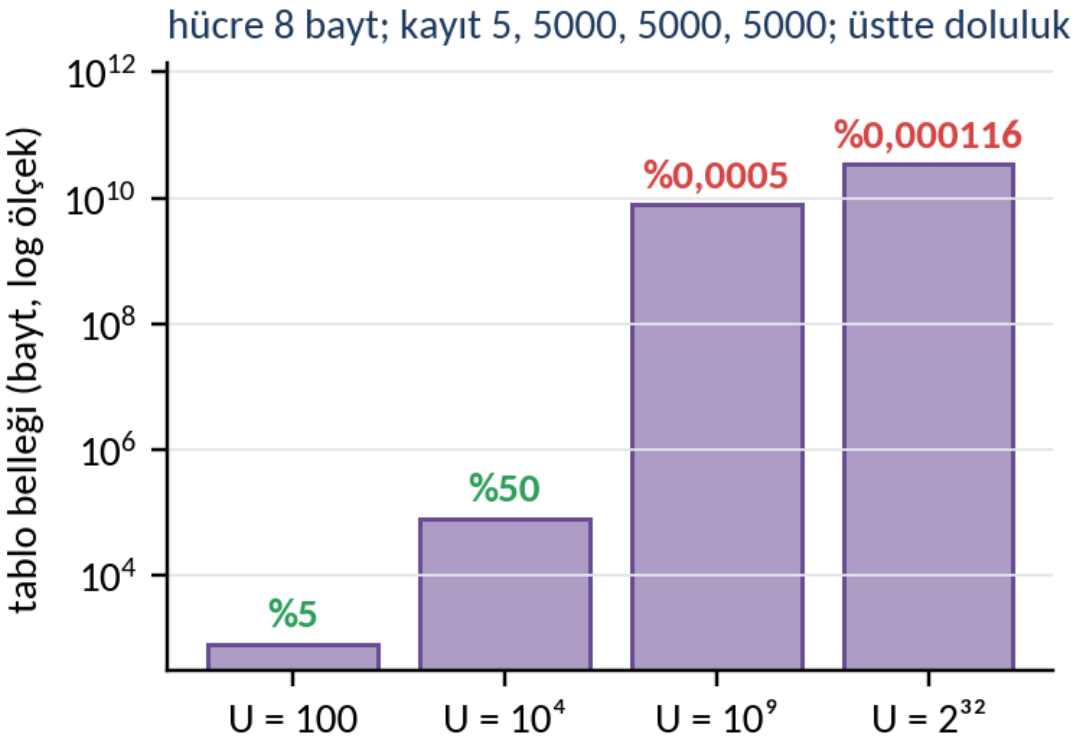
Sözlük ADT'si: ekle, ara, sil — dört gerçekleştirim

Güncelleme semantiği: anahtar zaten varsa değeri güncellenir; bir anahtar tabloda en çok bir kez bulunur. Yalnız anahtarlar: **küme**.

Gerçekleştirim	ara	ekle	sil	Koşul
Sırasız dizi	Θ(n)	Θ(n)	Θ(n)	yalnız eşitlik
Sıralı dizi + ikili arama (H4)	Θ(log n)	Θ(n)	Θ(n)	anahtarlar sıralanabilir
Doğrudan adresleme	Θ(1)	Θ(1)	Θ(1)	küçük tamsayı evreni 0 … U − 1
Karma tablosu	O(1) beklenen	O(1) amortize	O(1) beklenen	eşitlik + karma; en kötü Θ(n)

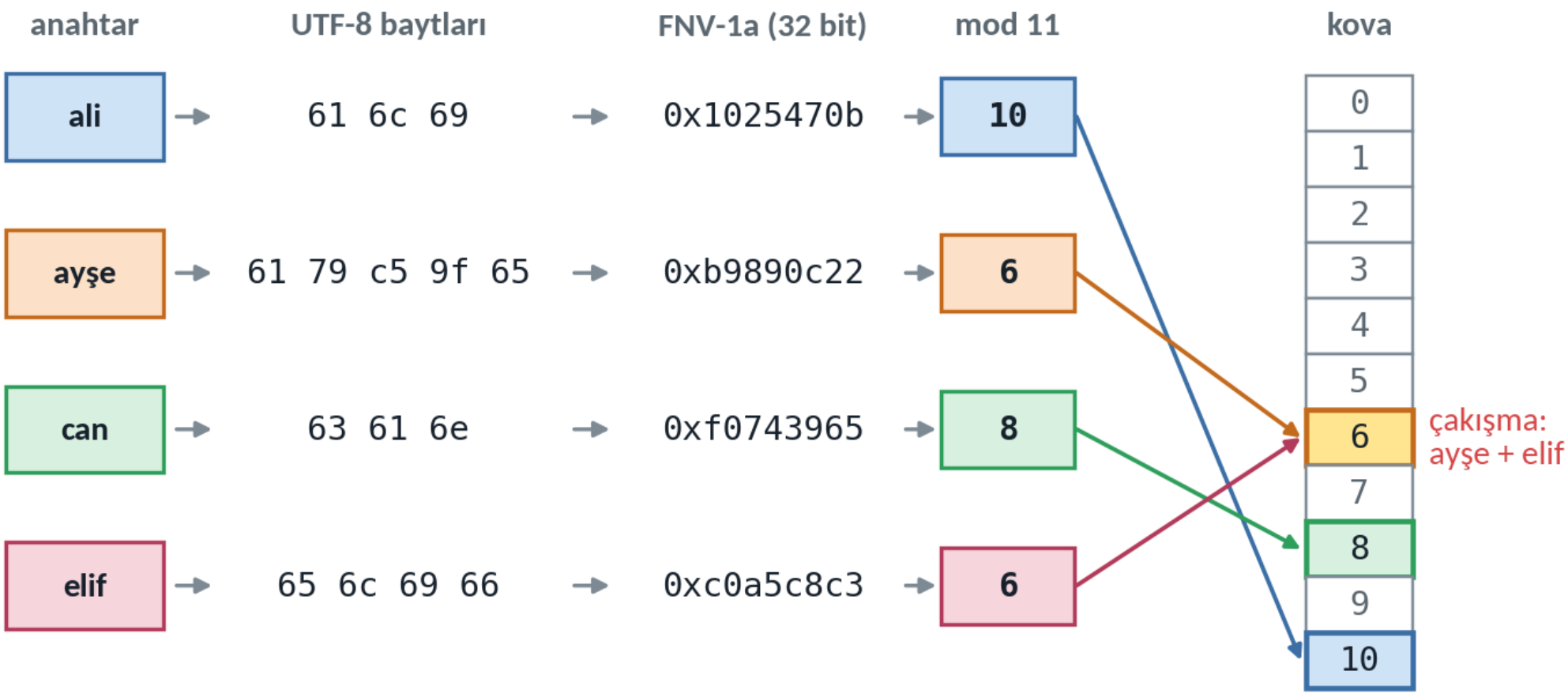
Doğrudan adresleme: anahtar = indis; anahtar evreni büyürse?





Örnek 1: 0–99 numaralar Θ(1); 9 haneli numarada tablo 8 GB, 5000 kayıtla doluluk %0,0005

Anahtar → baytlar → karma → kova



Örnek 3: UTF-8 baytlarından 32 bitlik FNV-1a, sonra mod 11 (bölme yöntemi, asal m); ayşe ve elif aynı kovada (çakışma)

TAHMİN ET

elif'in FNV-1a karması 0xc0a5c8c3 = 3 232 090 307. m = 11 kovalı tabloda hangi kovaya düşer?

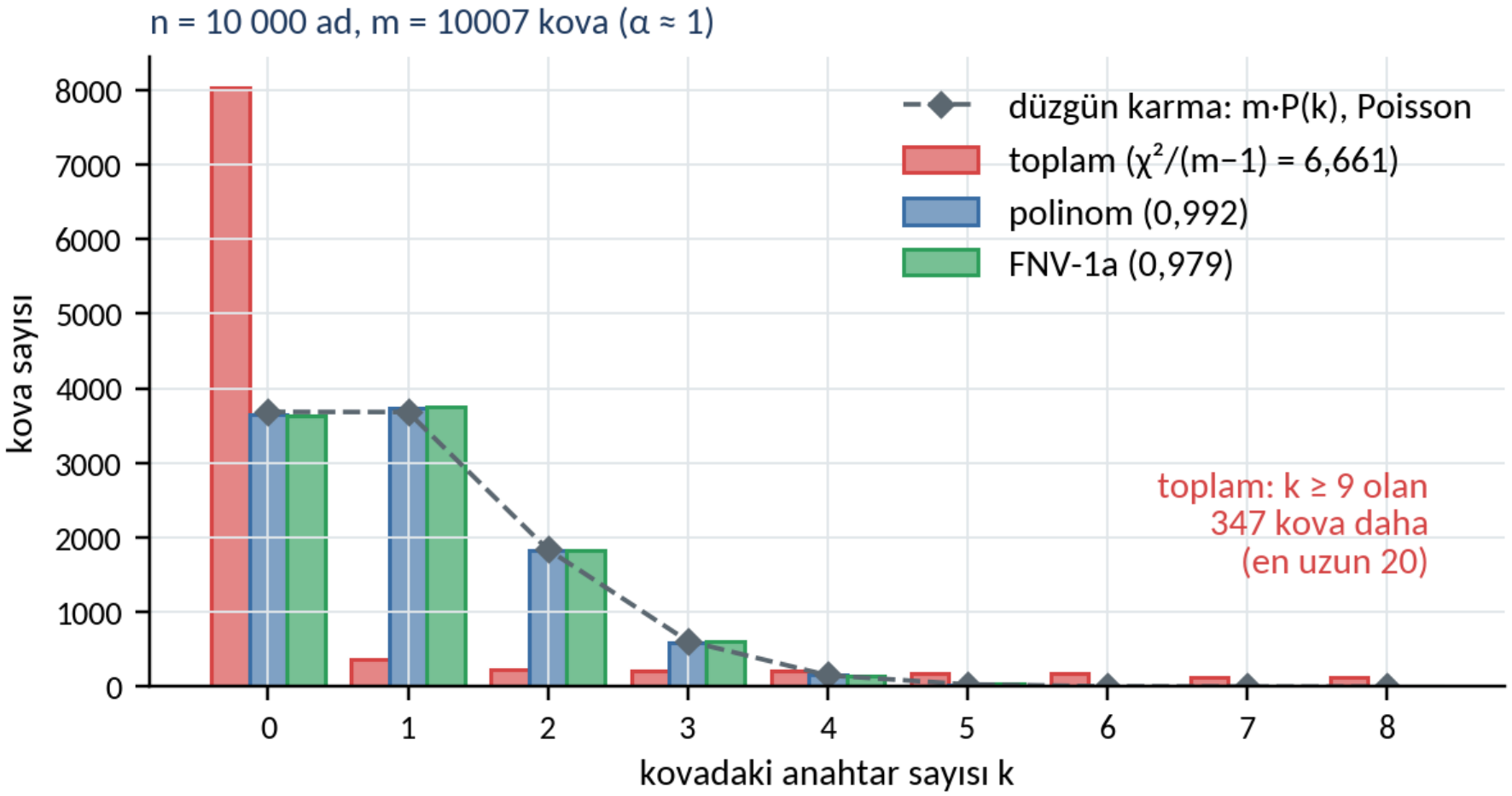
A kova 0

B kova 3

C kova 6

D kova 9

Karma fonksiyonları: toplam kötü, polinom ve FNV-1a düzgün



Örnek 2, 10 000 ad, m = 10 007: χ²/(m − 1) toplam 6,661, FNV-1a 0,979; boş kova 8 040 ↔ 3 630

Üç dilde: polinom karma (Horner, taban 31)

H10_02_karma_fonksiyonlari.c — C

```c
    uint32_t h = 0;
    for (; *s != '\0'; s++) {
        h = 31u * h + (unsigned char)*s;
    }
    return h;
```

H10_02_karma_fonksiyonlari.cpp — C++

```cpp
    std::uint32_t h = 0;
    for (unsigned char b : s) {
        h = 31u * h + b;
    }
    return h;
```

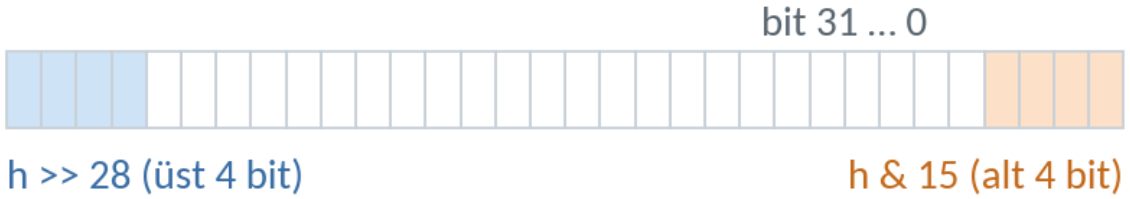
H10_02_karma_fonksiyonlari.py — Python

```python
    h = 0
    for b in s:
        h = (31 * h + b) & MASKE32
    return h
```

C/C++ `uint32_t` mod 2^{32} sarar; Python'da her adımda `& 0xFFFFFFFF` — üç dilde aynı karma (satırlar: C 36–40 · C++ 37–41 · Python 29–32)

Kaputun Altında: maske, düşük bitler, önbellek

h = 32 bit karma; m = 2^p = 16

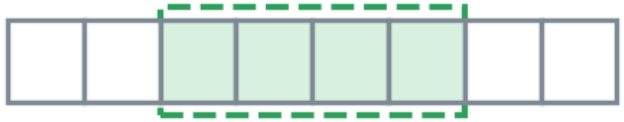


Kimlik karması h(k) = k ile 1024 düzenli anahtar → dolu kova (K01):

	m=16 maske	m=17 asal	m=16 çarpma
ardışık 0, 1, 2, ...	16	17	16
çift 0, 2, 4, ...	8	17	16
16k: 0, 16, 32, ...	1	17	16

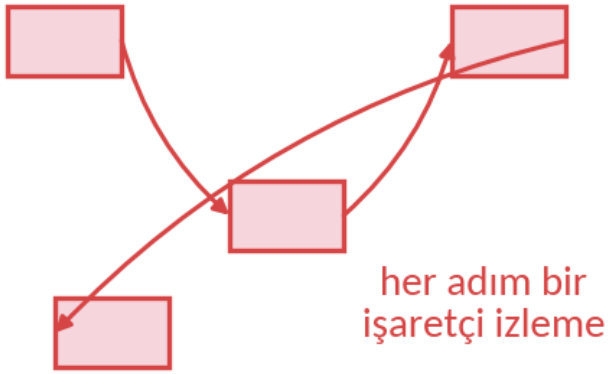
Bellekte yoklama

açık adresleme: bitişik hücreler



aynı önbellek satırı

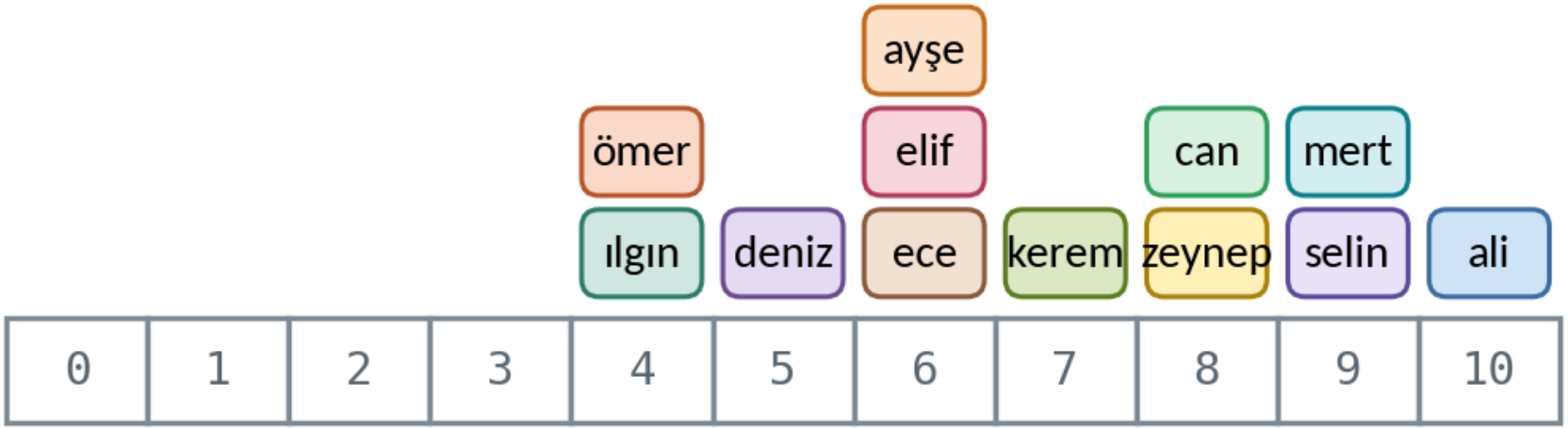
zincirleme: dağınık düğümler



$m = 2^p$: h % m = h & (m − 1), maske yalnız düşük bitleri görür; asal m (bölme) ya da çarpma yönteminin üst bitleri dağıtır

Çakışma kaçınılmazdır: güvercin yuvası ilkesi

Güvercin yuvası: n = 12 anahtar, m = 11 kova → en az bir kovada ⌈n/m⌉ = 2 anahtar



Örnek 3'ün 12 anahtarı (FNV-1a mod 11); boş kova 4, en uzun zincir 3

Dizgi evreni sınırsız, 32 bit karma değeri yalnızca 2^{32} tane:

çakışma kaçınılmazdır; soru, ne kadar seyrek ve nasıl çözüleceğidir

m kovaya m + 1 anahtar: en az bir kovada iki anahtar; dizgi sonsuz, 32 bitlik karma 2^{32}

TAHMİN ET

23 kişilik bir sınıfta en az iki kişinin doğum gününün aynı olma olasılığı yaklaşık kaçtır?

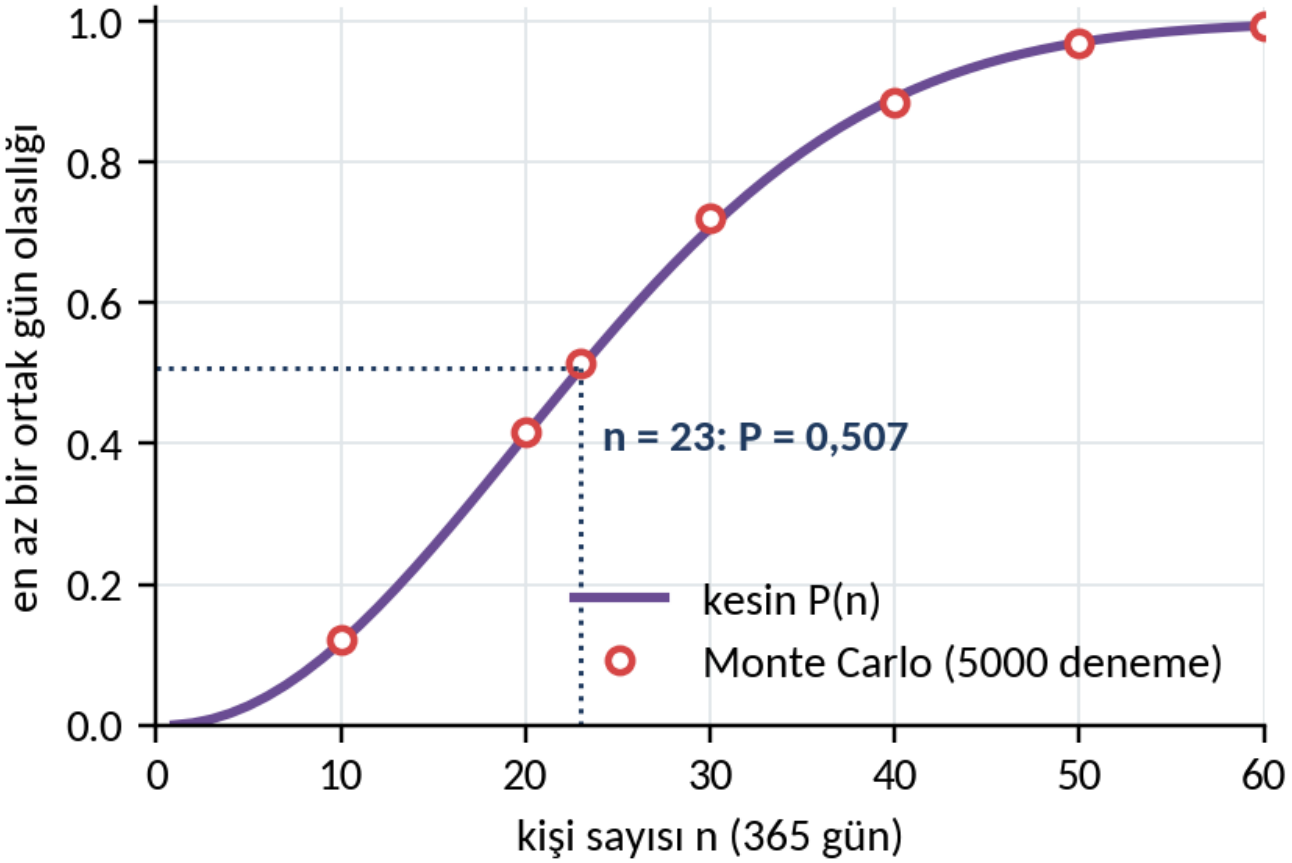
A ≈ %51 (0,507)

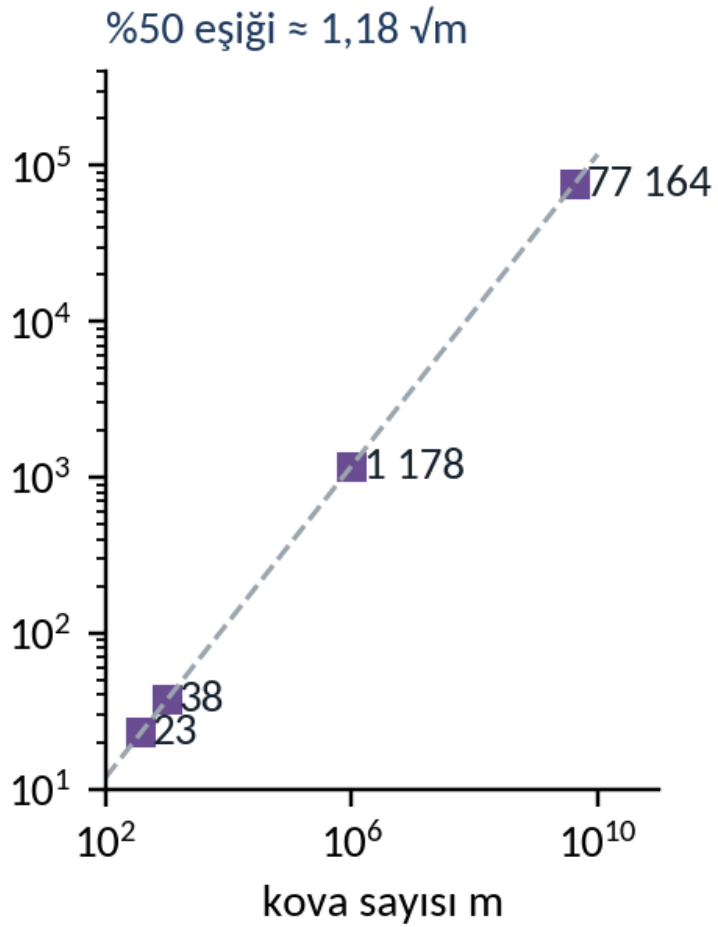
B ≈ %6 (23/365)

C ≈ %25

D ≈ %99

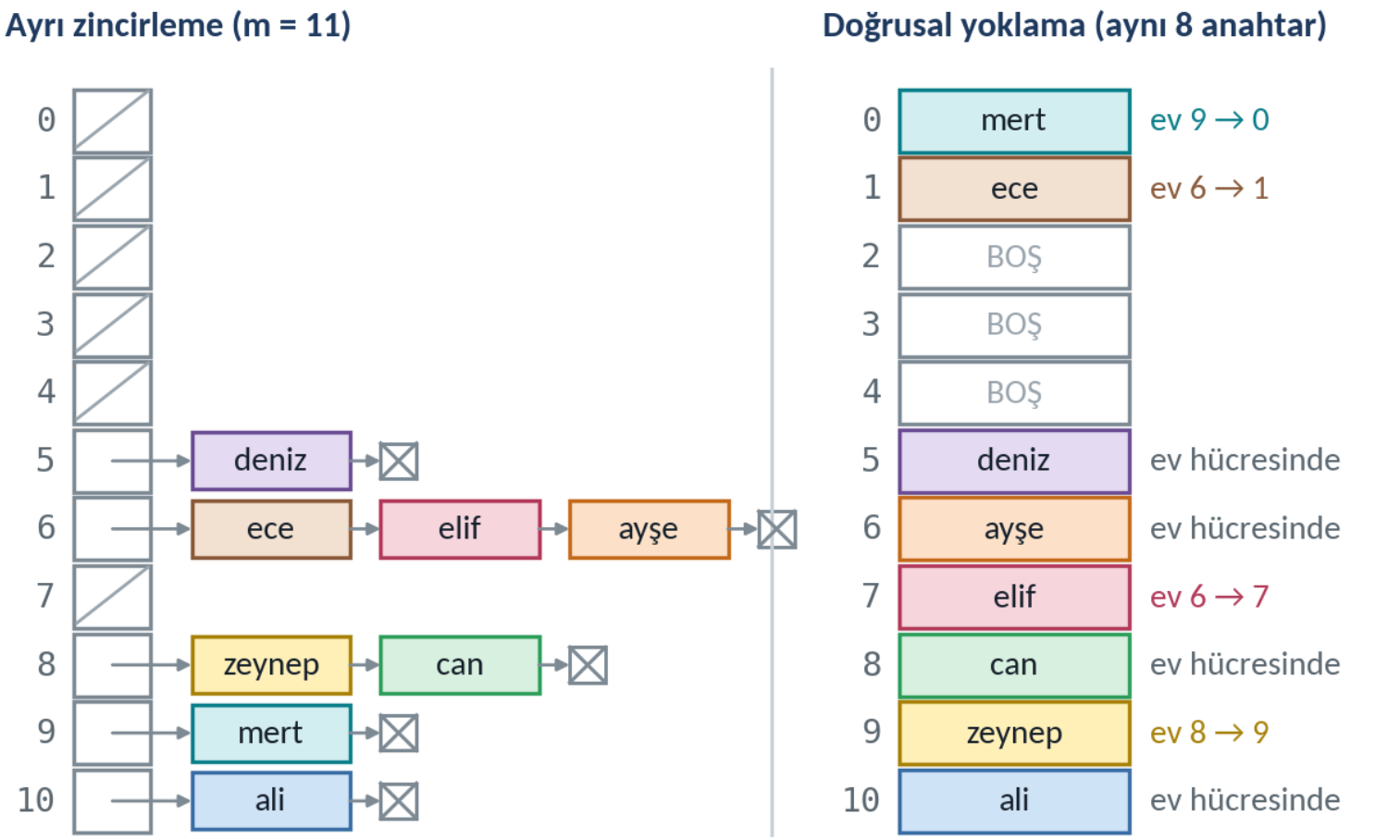
Doğum günü paradoksu ve karma: ilk çakışma ≈ √(πm/2)





%50 eşiği: m = 365'te 23, m = 10⁶'da 1 178, 32 bit karmada 77 164 anahtar (Örnek 10)

Çakışmayı çözmenin iki yolu: aynı 8 anahtar



Solda kova = bağlı liste (Örnek 3), sağda bir hücre = bir anahtar, doluysa sağa (Örnek 4); FNV-1a mod 11

TAHMİN ET

Kova 4'ün zinciri ılgın → ömer. Tabloda olmayan berk'in karması da kova 4'e düşüyor. `ara berk` kaç anahtar karşılaştırması yapar?

A 0: kova boş sayılır

B 2: zincirin tamamı

C 1: ilk düğümde biter

D 12: bütün tablo

Ayrı zincirleme: EKLE (güncelleme semantiği) ve zincirde arama

```
algoritma EKLE(T, k, v)
    j ← karma(k) mod m
    d ← T.kova[j]
    iken d ≠ BOŞ ve d.anahtar ≠ k
        d ← d.sonraki
    eğer d ≠ BOŞ ise
        d.değer ← v; döndür
    yeni ← düğüm(kopya(k), v)
    yeni.sonraki ← T.kova[j]
    T.kova[j] ← yeni; n ← n + 1
```

c/H10_03_ayri_zincirleme.c · satır 81–88

C

```c
    for (; d != NULL; d = d->sonraki) {
        karsilastirma_sayisi++;
        if (strcmp(d->anahtar, anahtar) == 0) {
            return d;
        }
    }
    return NULL;
}
```

Beklenen O(1 + α): başarısız arama α, başarılı ≈ 1 + α/2 düğüm · ekleme zincirin başına O(1)

Üç dilde: zincirde arama

H10_03_ayri_zincirleme.c — C

```c
    for (; d != NULL; d = d->sonraki) {
        karsilastirma_sayisi++;
        if (strcmp(d->anahtar, anahtar) == 0) {
            return d;
        }
    }
    return NULL;
```

H10_03_ayri_zincirleme.cpp — C++

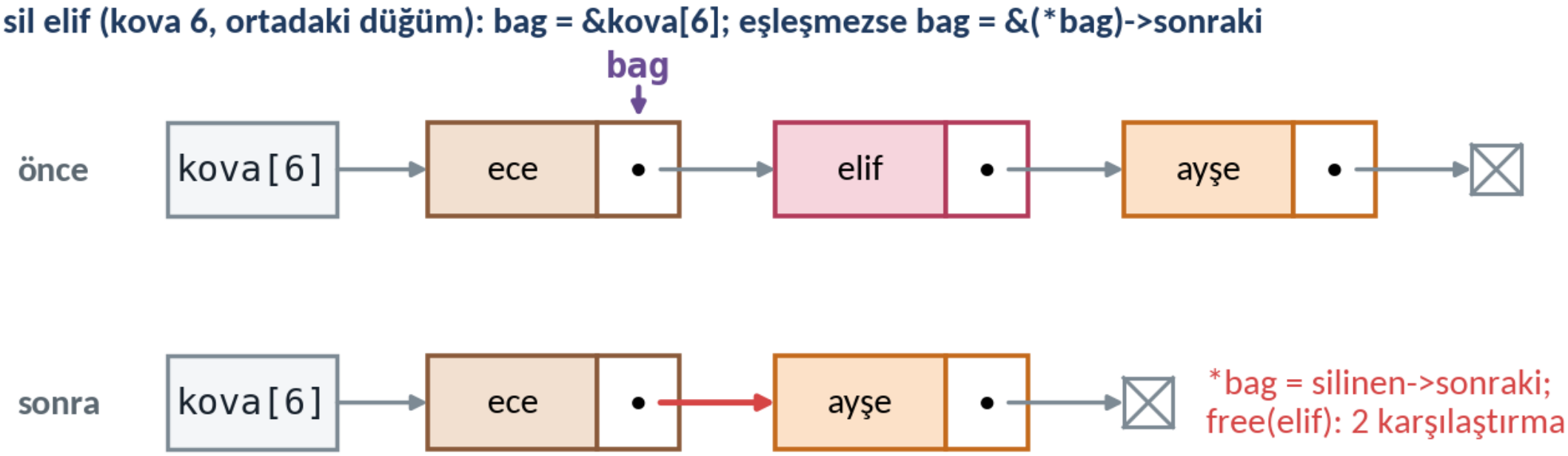
```cpp
    for (; d != nullptr; d = d->sonraki) {
        karsilastirma_sayisi++;
        if (d->anahtar == anahtar) {
            return d;
        }
    }
    return nullptr;
```

H10_03_ayri_zincirleme.py — Python

```python
    global karsilastirma_sayisi
    while d is not None:
        karsilastirma_sayisi += 1
        if d.anahtar == anahtar:
            return d
        d = d.sonraki
    return None
```

C `strcmp`, C++ `std::string` ==, Python `bytes` ==; düğüm C'de `Dugum *`, Python'da nesne başvurusu (satırlar: C 81–87 · C++ 61–67 · Python 50–56)

Zincirden silme: Dugum ** ile özel durumsuz



*bag = önceki düğümün sonraki alanının adresi; *bag = silinen->sonraki, sonra free — baştaki düğüm için de aynı kod*

ALGORİTMA KİMLİK KARTI

Karma tablosu: ekle / ara / sil *hash table: insert / search / delete*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Anahtar → karma → kova indisi	Eşitlik + karma fonksiyonu	O(1)	O(1) beklenen	Θ(n)	O(n)

Özellikler	Ortalama süre karma kalitesine ve α'ya bağlı; en kötü durumda tek kova (saldırı)
Değişmez / doğruluk	Her anahtar, karma değerinin belirlediği kovadadır (zincirlemede o kovanın listesinde; açık adreslemede ev hücresinden başlayan yoklama dizisinde)
Kütüphane karşılığı	C: yok (elle; POSIX `hsearch`: süreç başına tek tablo) · C++: `std::unordered_map`, `std::unordered_set` · Python: `dict`, `set`
Ne zaman kullanılır	Eşitlik araması: frekans, yinelenen, iki toplam, anagram
Ne zaman kullanılmaz	Sıralı dolaşma, aralık sorgusu, en küçük/en büyük

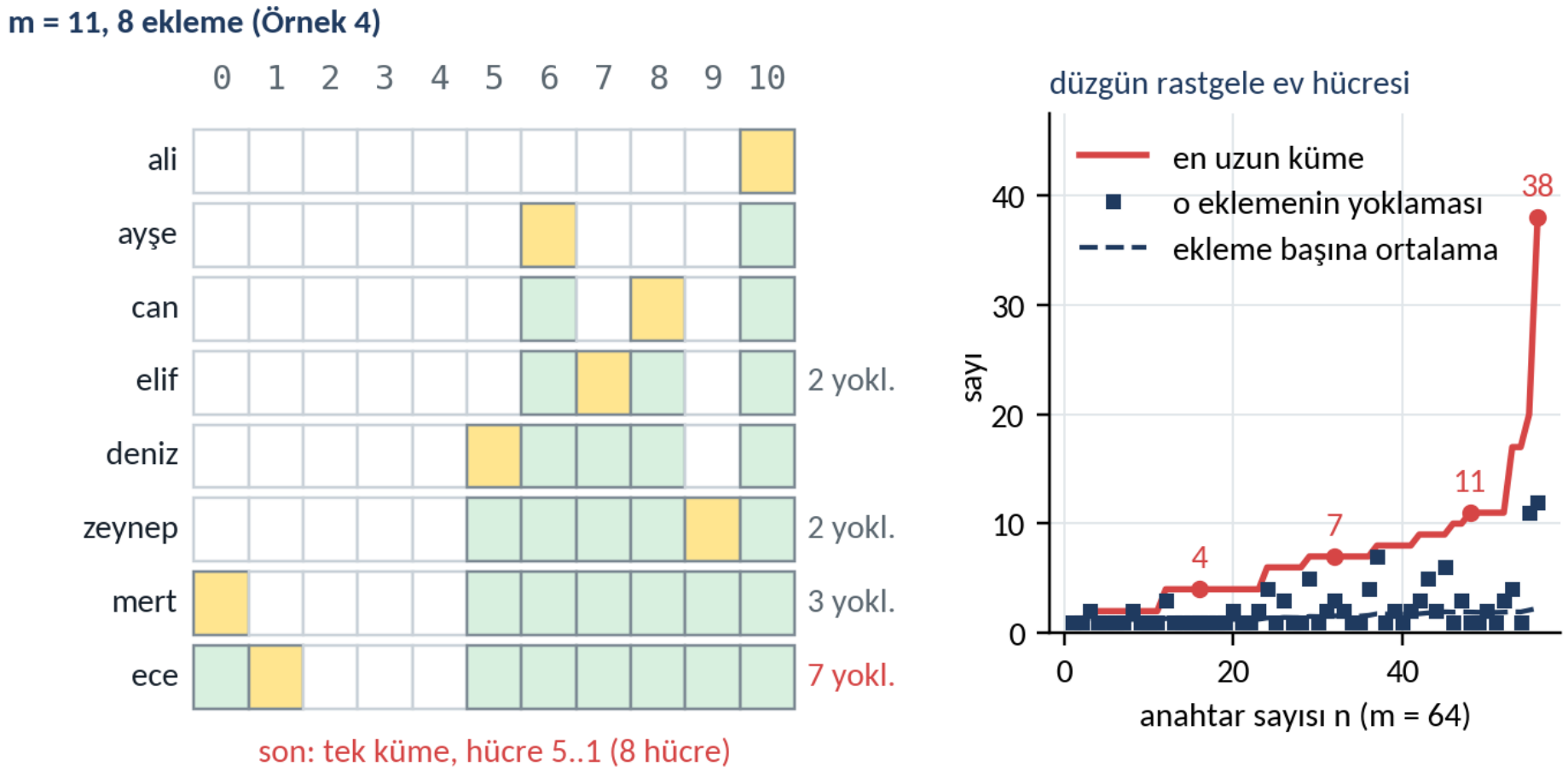
ALGORİTMA KİMLİK KARTI

Ayrı zincirleme *separate chaining*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Her kova bir bağlı liste; başa ekleme	Karma fonksiyonu; anahtar kopyalanır	O(1)	O(1 + α)	Θ(n)	O(n + m)

Özellikler	α > 1 olabilir; silme kolay; her düğüm ayrı ayırma ve işaretçi izleme
Değişmez / doğruluk	j kovasının listesindeki her anahtar için karma(anahtar) mod m = j; bir anahtar tabloda en çok bir kez bulunur
Kütüphane karşılığı	C: yok (elle) · C++: `std::unordered_map` (düğüm tabanlı kovalar; `bucket_count`, `bucket_size`) · Python: yok — `dict` açık adreslemelidir
Ne zaman kullanılır	Sık silme, α > 1 olabilir, düğüm adresi kararlı kalmalı
Ne zaman kullanılmaz	Küçük anahtarla çok sık arama (önbellek kaçırma)

Doğrusal yoklama ve birincil kümelenme



m = 64, düzgün rastgele ev: en uzun küme α = 0,25'te 4, α = 0,75'te 11, n = 56'da 38

TAHMİN ET

can silindi ve hücre 8 BOŞ yapıldı (mezar taşı yok). Ev hücresi 6 olan ece (hücre 1'de) aranırsa arama nerede durur?

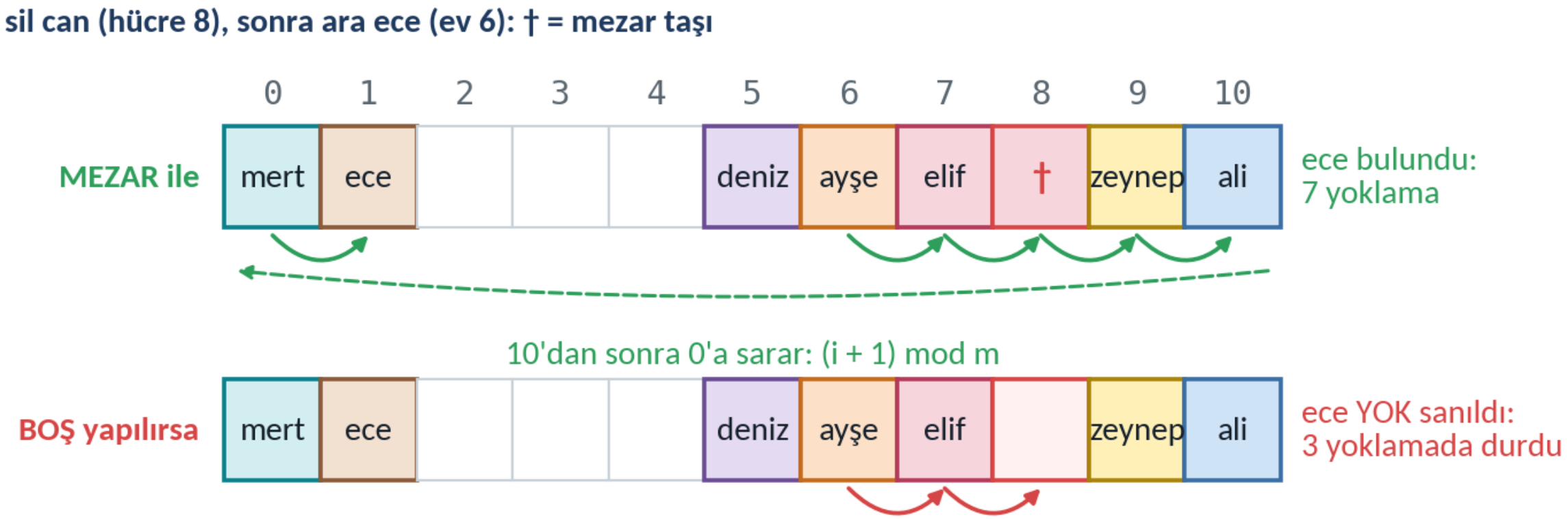
A Hücre 1'de: ece bulunur

B Hücre 10'da: tablonun sonunda

C Hücre 0'da: mert'te

D Hücre 8'de: BOŞ, “yok” (3 yoklama)

Mezar taşı: arama durmaz, ekleme yeniden kullanır



ara ece: ev 6, yoklama 6 7 8 9 10 0 1 (7) -> 80 · ekle kerem: ev 7, yoklama 7 8 9 10 0 1 2 (7) -> hücre 8 (ilk mezar taşı)

C: doğrusal yoklamada arama döngüsü (Örnek 4)

c/H10_04_dogrusal_yoklama.c · satır 105–117

C

```c
        const Hucre *h = &t->hucre[i];
        if (h->durum == BOS) {
            *yer = ilk_mezar >= 0 ? ilk_mezar : (long)i;
            return -1;
        }
        if (h->durum == MEZAR) {
            if (ilk_mezar < 0) {
                ilk_mezar = (long)i;
            }
        } else if (strcmp(h->anahtar, anahtar) == 0) {
            return (long)i;
        }
        i = (i + 1) % t->m;                    /* doğrusal yoklama */
```

BOŞ Aramayı durdurur; eklenecek yer: ilk mezar taşı, yoksa bu hücre.

MEZAR Atlanır, ama ilk mezar taşı hatırlanır (ekleme yeniden kullanır).

DOLU `strcmp` ile karşılaştırılır; döngü en çok m adım (Hata Avcısı 4).

Üç dilde: mezar taşıyla silme

C

```c
long yer;
long i = bul(t, anahtar, &yer);
if (i < 0) {
    return 0;
}
free(t->hucre[i].anahtar);
t->hucre[i].anahtar = NULL;
t->hucre[i].durum = MEZAR;
t->dolu--;
t->mezar++;
return 1;
```

C++

```cpp
long yer;
long i = bul(t, anahtar, yer);
if (i < 0) {
    return false;
}
t.hucre[i].anahtar.clear();
t.hucre[i].durum = MEZAR;
t.dolu--;
t.mezar++;
return true;
```

Python

```python
i, _ = bul(t, anahtar)
if i < 0:
    return False
t.hucre[i].anahtar = None
t.hucre[i].durum = MEZAR
t.dolu -= 1
t.mezar += 1
return True
```

Silinen hücre BOŞ değil MEZAR olur; sayaçlar: dolu − 1, mezar + 1 (yük denetimi ikisine birlikte bakar) (H10_04_dogrusal_yoklama · satırlar: C 164–174 · C++ 126–135 · Python 99–106)

ALGORİTMA KİMLİK KARTI

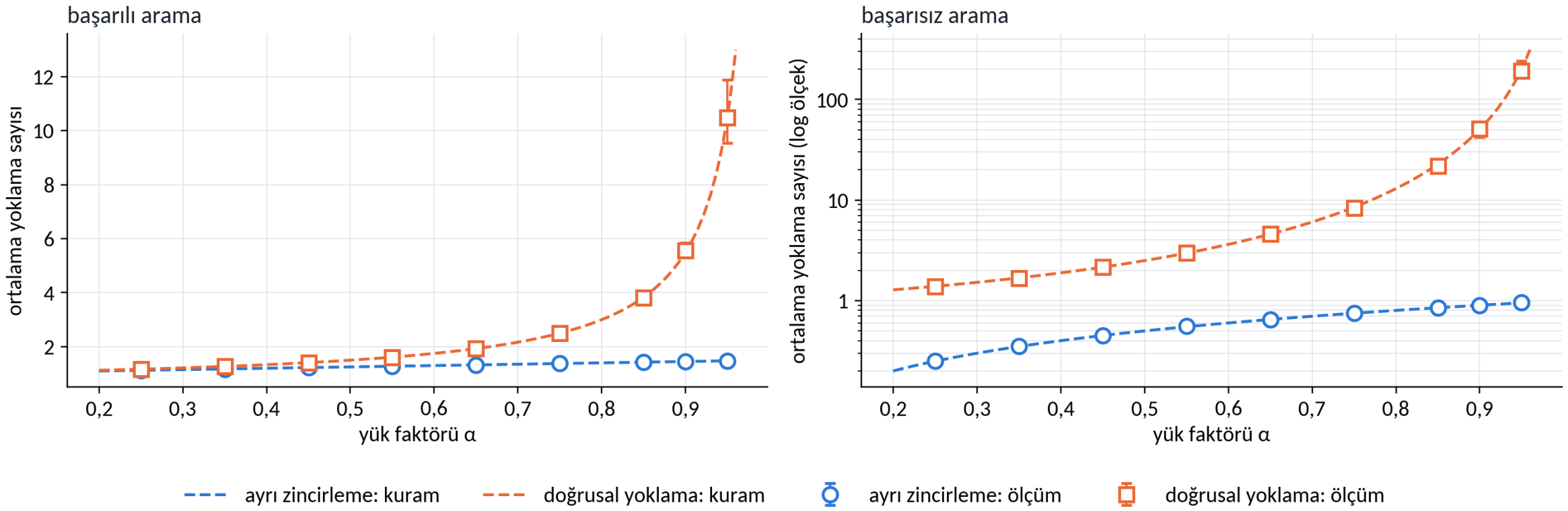
Doğrusal yoklama (açık adresleme) *linear probing, open addressing*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Tek dizi; doluysa i + 1 (mod m)	$\alpha < 1$ (yük denetimi)	$O(1)$	$\approx \frac{1}{2}(1 + 1/(1-\alpha)^2)$ yoklama	$\Theta(n)$	$O(m)$

Özellikler	Birincil kümelenme; bitişik bellek önbellek dostu; silmede mezar taşı
Değişmez / doğruluk	Bir anahtarın ev hücresinden bulunduğu hücreye kadarki yoklama dizisinde BOŞ hücre yoktur (MEZAR olabilir)
Kütüphane karşılığı	C: yok (elle) · C++: yok (standart kütüphanede açık adreslemeli tablo yok) · Python: `dict`/`set` açık adreslemelidir (yoklama dizisi doğrusal değil, perturb ile)
Ne zaman kullanılır	Küçük anahtar/değer, sık arama, $\alpha \leq$ ~0,7
Ne zaman kullanılmaz	$\alpha \to 1$, çok silme, kötü karma

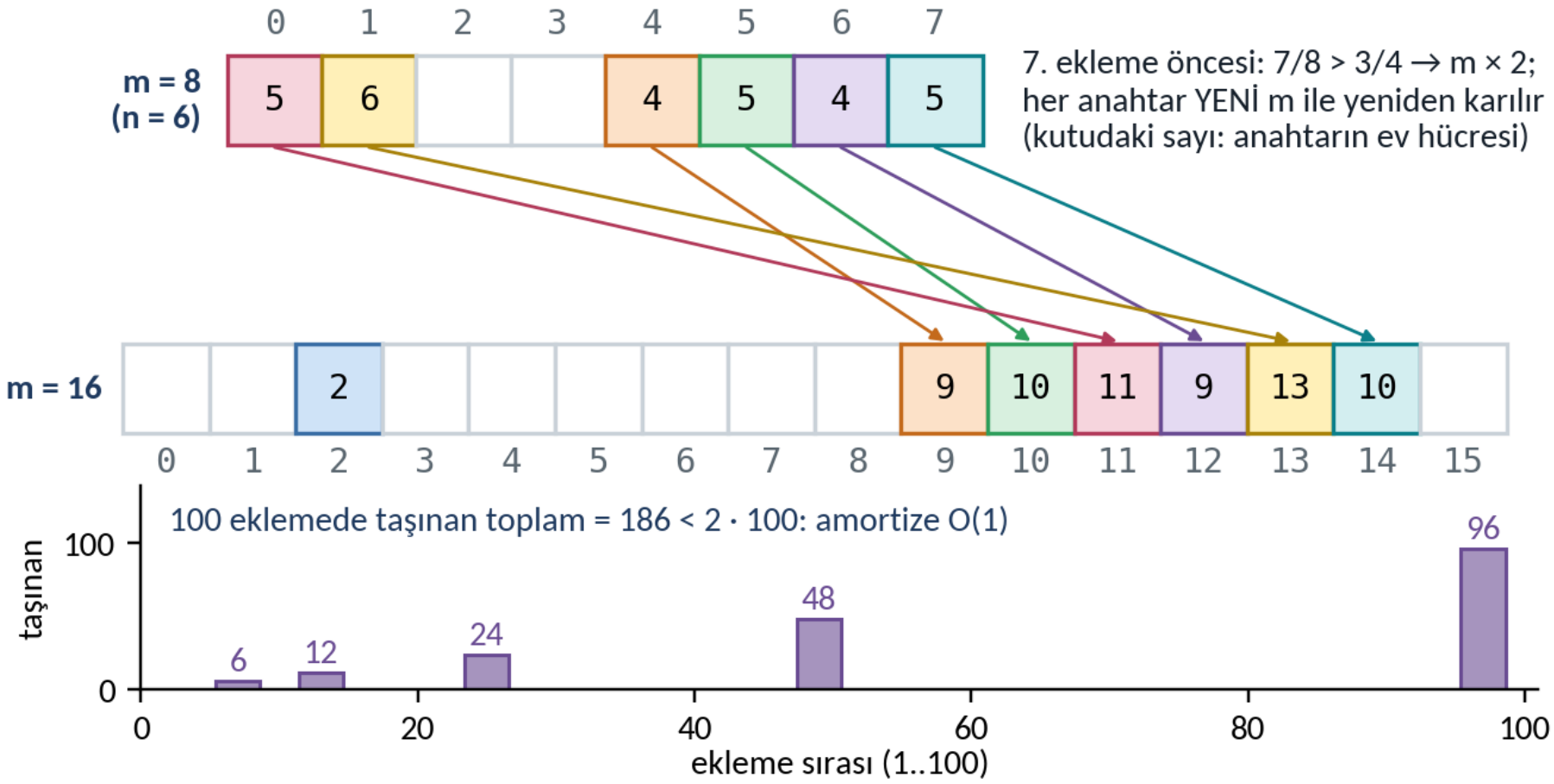
Yük faktörü → yoklama: kuram ve ölçüm

m = 65536, 7 tohum; nokta = ortalama, çubuk = en az–en çok; kesikli = Knuth (1998) yaklaşıkları



m = 2^{16}, 7 tohum: zincirleme kurama ≤ %1,2 uyar; doğrusal başarısız α = 0,95'te 192,3 (kuram 200,5)

Yeniden boyutlandırma: m × 2, her anahtar yeniden karılır



100 eklemeden sonra: m = 256, α = 0,391, toplam taşınan = 186 · indis m'ye bağlı: kopyalamak yetmez (H8'den farkı)

TAHMİN ET

Sıradaki büyütme m = 256 → 512. Hangi eklemeden önce olur ve kaç anahtar taşınır? (eşik: (n + 1)/m > 3/4)

A 193. eklemeden önce, 192 anahtar

B 257. eklemeden önce, 256 anahtar

C 192. eklemeden önce, 191 anahtar

D 129. eklemeden önce, 128 anahtar

Yük denetimli ekleme ve büyütme

```
algoritma EKLE(T, k)
    eğer 4·(T.n + 1) > 3·T.m ise BÜYÜT(T)
    YERLEŞTİR(T, k)
algoritma BÜYÜT(T)
    Y ← boş tablo, kapasite 2·T.m
    her dolu hücre i için
        YERLEŞTİR(Y, T[i].anahtar)
    T ← Y
```

c/H10_06_yeniden_boyutlandirma.c · satır 106–115

C

```c
static int tablo_ekle(Tablo *t, uint32_t k)
{
    size_t m = (size_t)1 << t->p;
    if (4 * (t->n + 1) > 3 * m) {
        if (buyut(t) != 0) {
            return -1;
        }
    }
    return yerlestir(t, k);
}
```

Tamsayı yük denetimi (kayan nokta yok) · büyütme başarısızsa eski tablo değişmez (−1)

ALGORİTMA KİMLİK KARTI

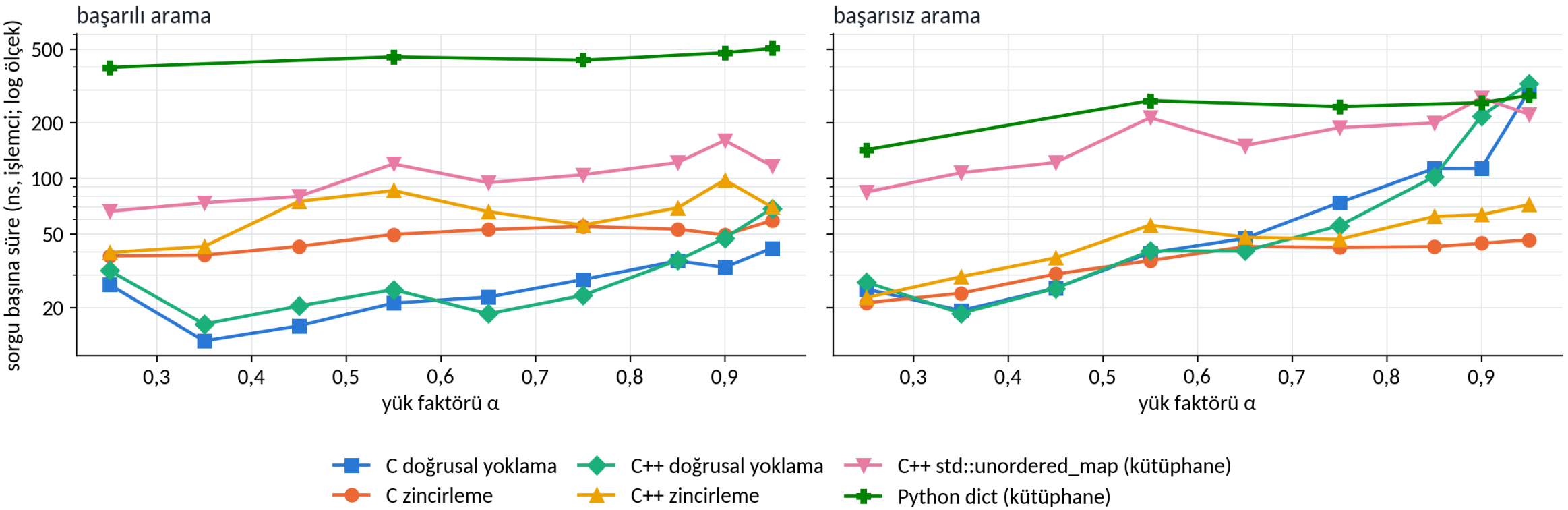
Yeniden boyutlandırma *resizing, rehashing*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Eşik aşılacaksa m × 2, yeniden karma	Bir yük eşiği (3/4)	—	**O(1) amortize**	**Θ(n) tek işlem**	**O(n)**

Özellikler	N eklemede taşınan < 2N; tek büyütme Θ(n) duraklama
Değişmez / doğruluk	Her eklemeden sonra n/m ≤ 3/4; o ana dek taşınanların toplamı < 2N
Kütüphane karşılığı	C: yok (elle; `realloc` değil, yeni tablo + yeniden karma) · C++: `rehash`, `reserve`, `max_load_factor` (varsayılan 1,0) · Python: `dict` kendiliğinden büyür (CPython: kullanılabilir oran 2/3)
Ne zaman kullanılır	n önceden bilinmiyorsa
Ne zaman kullanılmaz	Tek Θ(n) duraklama kabul edilemiyorsa (artımlı büyütme)

Aynı yoklama, farklı süre: bitişik hücre ↔ dağınık düğüm

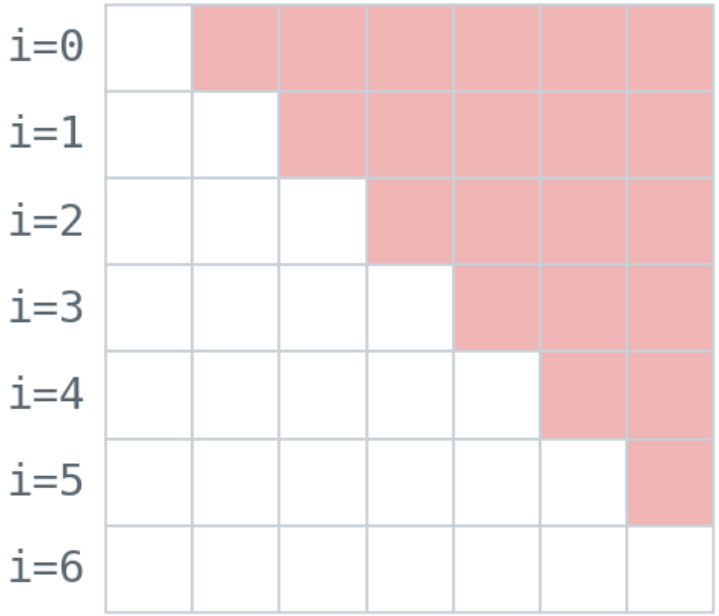
m = 1048576 hücre/kova; sorgular karıştırılmış sırada; 1 ısınma + 5 tekrar, ortanca (bu makinede)



m = 2²⁰, bu makinede: başarılı aramada zincirleme α ≤ 0,75'te 1,43–2,91 kat yavaş; α = 0,95 başarısızda oran 0,16 (sıra döner)

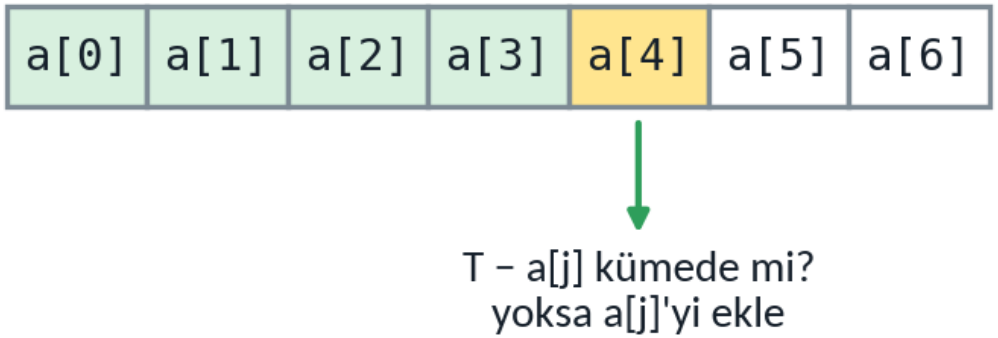
Uygulamalar: sayaç, küme, harita — Θ(n²) → beklenen Θ(n)

Kaba kuvvet: her çift (i < j)



n = 7: n(n−1)/2 = 21 karşılaştırma

Karma kümesi: tek geçiş



n = 7: 15 yoklama (öğe başına ≤ 2 işlem)

n = 10^5 yinelenen tespiti: 107 133 yoklama (öğe başına 1,071)

kaba kuvvet: n(n−1)/2 = 4 999 950 000 karşılaştırma

Örnek 7: 6 248 sözcükte 3 277 farklı sözcük (sayaç) · Örnek 8: yinelenen ve iki toplam (küme, harita) · Örnek 9: 16 anagram grubu

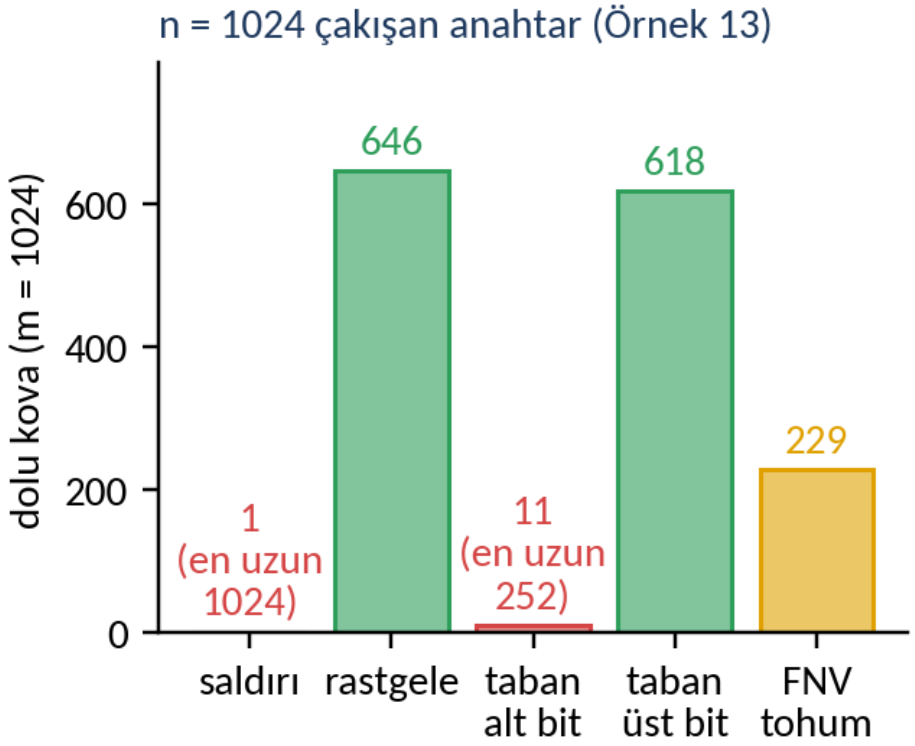
Karma taşkını: herkesçe bilinen formül, tek kova

polinom 31: h("Aa") = h("BB") = 2112

eşit uzunlukta bloklar: h(xy) = h(x)·31^|y| + h(y)

Aa	Aa	Aa	Aa	→ 3754541312
Aa	Aa	Aa	BB	→ 3754541312
Aa	Aa	BB	Aa	→ 3754541312
Aa	Aa	BB	BB	→ 3754541312
Aa	BB	Aa	Aa	→ 3754541312
Aa	BB	Aa	BB	→ 3754541312
Aa	BB	BB	Aa	→ 3754541312
Aa	BB	BB	BB	→ 3754541312

… k = 4 blok: 2^4 = 16 anahtar, hepsi aynı karma



h("Aa") = 65·31 + 97 = 2112, h("BB") = 66·31 + 66 = 2112 · n = 1024'te saldırı 1 kova, rastgele anahtar 646 kova

TAHMİN ET

Saldırı anahtarlarıyla n = 512'den 1024'e çıkınca toplam anahtar karşılaştırması kaç katına çıkar? (her ekleme, anahtar zaten var mı diye zinciri tarar)

A ≈ 2 kat (doğrusal)

B ≈ 4 kat (karesel)

C ≈ 8 kat (kübik)

D ≈ 1 kat (değişmez: hep aynı kova)

C: 2^k çakışan anahtar ve dört savunma

c/H10_13_karma_saldirisi.c · satır 213–221

C

```c
213  static void carpisan_anahtar(size_t i, int k, char *s)
214  {
215      for (int b = 0; b < k; b++) {
216          const char *blok = (i >> (k - 1 - b)) & 1u ? "BB" : "Aa";
217          s[2 * b] = blok[0];
218          s[2 * b + 1] = blok[1];
219      }
220      s[2 * k] = '\0';
221  }
```

KARESEL

Tohumlu h_0

Eşit uzunluklu her anahtara aynı $h_0 \cdot 31^L$ eklenir:
n = 1024'te yine 523 776 karşılaştırma.

GÜVENİLMEZ

Rastgele taban + alt bit

Alt t + s bit yalnız BB sayısını taşır. Bu tabanda 11 kova, n × 2'de 3,69 kat; Kavram 9: 2/20 tabanda karesele yakın.

DOĞRUSAL

Taban + üst bit; FNV + tohum

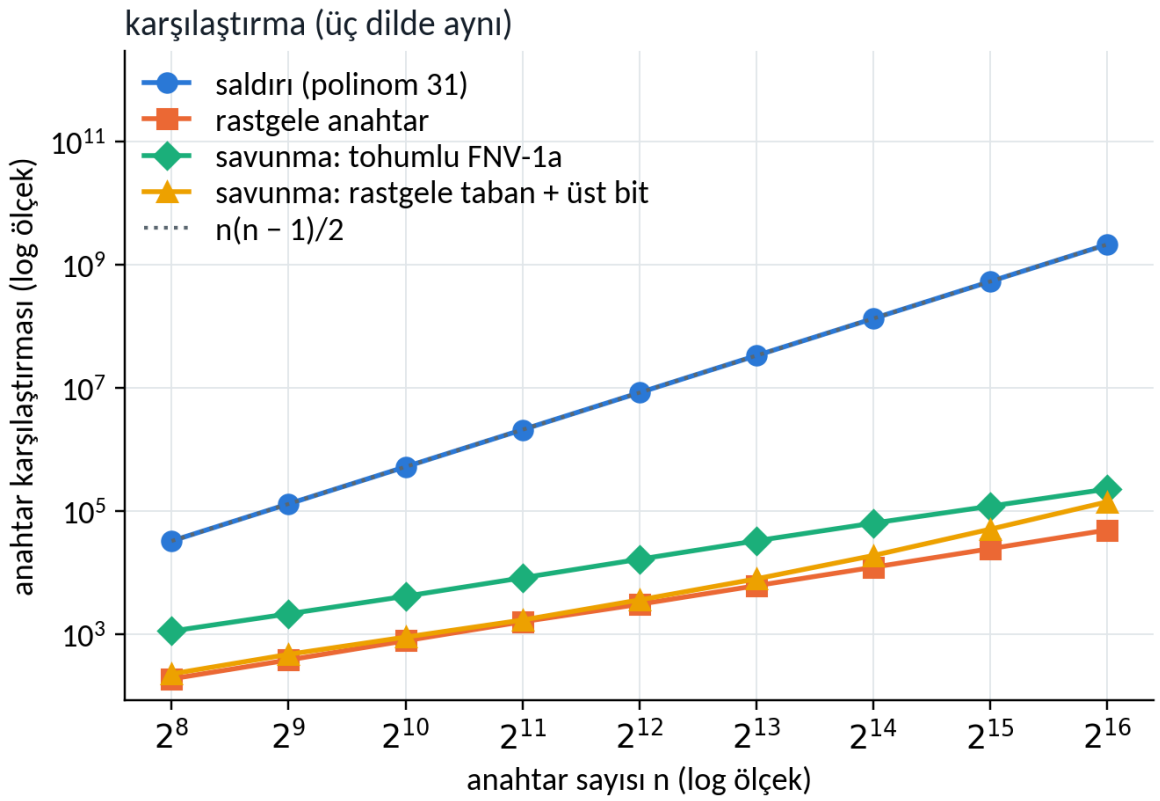
618 ve 229 kova dolu; 896 ve 4 133 karşılaştırma.

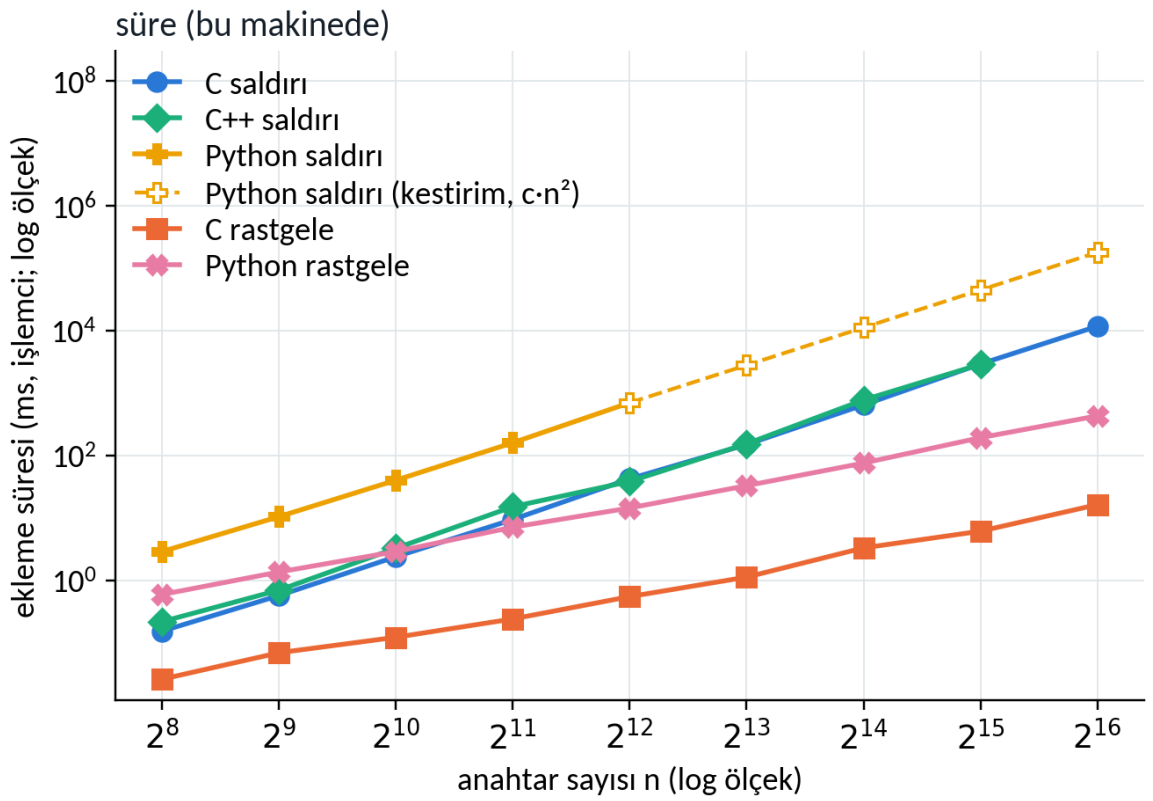
GERÇEK SAVUNMA

Anahtarlı karma (SipHash)

128 bit gizli anahtar; Python 3.4'ten beri `str`/`bytes`. Kriptografik karma (SHA-256) tablo için yavaş.

Saldırı altında n ekleme: karesel büyüme ve savunmalar





log–log eğim: saldırı C 2,04, Python 1,98 (≈ n²); n = 2¹⁶'da C'de saldırı rastgele anahtardan 718,2 kat yavaş

Desen eşleme: naif yöntem her kaymayı dener



abracadabra / abra: 2 eşleşme, 16 karşılaştırma · en kötü a^{10000} / $a^{99}b$: 990 100 = (n − m + 1)·m

ALGORİTMA KİMLİK KARTI

Naif desen eşleme *naive string matching*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Her kayma s için P ile T[s..s+m−1] soldan sağa	Yok	O(n)	O(n) beklenen (rastgele metin)	O(n·m)	O(1)

Özellikler	Rastgele metin, m = 8: kayma başına 1,333; en kötü (n − m + 1)·m = 990 100
Değişmez / doğruluk	s'den küçük her kayma denenmiş, eşleşenler yazılmıştır
Kütüphane karşılığı	C: `strstr` (algoritması gerçekleştirime bağlı) · C++: `std::string::find`, `std::search` · Python: `str.find`, `bytes.find`, `in`
Ne zaman kullanılır	Kısa desen, tek seferlik arama, rastgele benzeri metin
Ne zaman kullanılmaz	Tekrarlı desen ve metin (aa…ab), en kötü durum güvencesi

TAHMİN ET

Rabin–Karp'ta sahte eşleşme var mı? T = 23590231415267399921, P = 31415, q = 13: P mod 13 = 7 ve s = 12'deki pencere 67399 mod 13 = 7. Doğrulamalı

A s = 6 ve s = 12 (iki karma eşitliği)

B Hiçbirini: q = 13 çok küçük

C Yalnız s = 12

D Yalnız s = 6; s = 12 sahte, doğrulamada elenir

Rabin–Karp: kayan karma O(1)'de güncellenir

T = 23590231415267399 21, P = 31415, d = 10, q = 13: p = P mod q = 7, h = d^(m−1) mod q = 3



t_s	8	9	3	11	0	1	**7**	8	4	5	10	11	**7**	9	11	(s = 0..14)		

Kayan güncelleme (kesikli pencere s = 0 → 1):

```
t_(s+1) = (d · (t_s + q − T[s]·h mod q) + T[s+m]) mod q
```

```
t_1 = (10·(8 + 13 − 6) + 2) mod 13 = 9   (çıkan 2, giren 2; O(1) iş)
```

yeşil: s = 6 gerçek eşleşme · kırmızı: s = 12 sahte eşleşme (67399 mod 13 = 7)

h = d^(m−1) mod q = 3; t_1 = (10·(8 + 13 − 6) + 2) mod 13 = 9 · yeşil: gerçek eşleşme, kırmızı: sahte eşleşme

C: kayan karma ve doğrulama (Örnek 12)

c/H10_12_rabin_karp.c · satır 70–79

C

```c
70      for (size_t s = 0;; s++) {
71          if (hp == ht) {
72              dogrula(t, s, p, m, r);
73          }
74          if (s + m >= n) {
75              break;
76          }
77          /* + q: C'de % bölünenin işaretini alır; ara değer negatif olmasın */
78          ht = (d * (ht + q - t[s] * h % q) + t[s + m]) % q;
79      }
```

ÖRNEK 12 BÖLÜM 3 (ÖZET)

d = 256, n = 10^5, m = 8: karma eşitliği / sahte
q = 13 7578 / 7577
q = 1000003 1 / 0
mod 2^{32} 406 / 405 ($256^4 \equiv 0$)

ALGORİTMA KİMLİK KARTI

Rabin–Karp *rolling hash*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kayan karma O(1)'de; karma eşitse doğrula	d ve q seçimi; her eşitlik doğrulanır	O(n + m)	O(n + m) beklenen	O(n·m)	O(1)

Özellikler	n = 10^5, m = 8, d = 256: q = 13'te 7 577 sahte, q = 1000003'te 0; mod 2^{32}'de 405 (Örnek 12)
Değişmez / doğruluk	Her kaymada t_s = (Σ T[s+i]·d^(m−1−i)) mod q
Kütüphane karşılığı	C: yok (elle) · C++: yok (elle) · Python: yok (elle)
Ne zaman kullanılır	Aynı uzunlukta çok desen (karmalar bir kümede), yinelenen blok/intihal
Ne zaman kullanılmaz	Modül küçük ya da sabit ve girdi saldırganın elindeyse

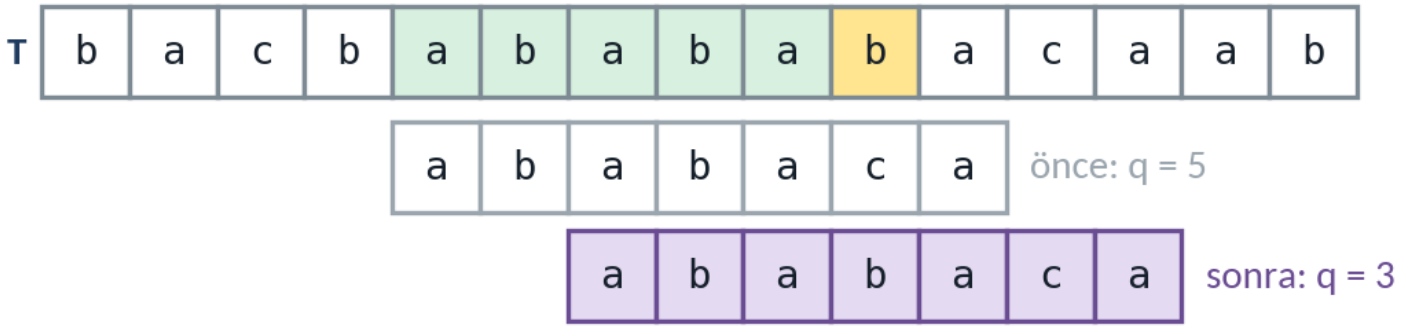
Merak: KMP — metinde geri dönmeden

Önek fonksiyonu π[q]: P[0..q−1]'in hem öneki hem soneki olan en uzun (kendisinden kısa) parça

q	1	2	3	4	5	6	7
P	a	b	a	b	a	c	a
π[q]	0	0	1	2	3	0	1

π[5] = 3: “ababa” → “aba”

Eşleme, i = 9: 5 karakter eşleşmişti; T[9] = 'b' ≠ 'c' (desenin 6. karakteri) → q = π[5] = 3



P = ababaca: π = [0, 0, 1, 2, 3, 0, 1] · karşılaştırma: π = 8 (≤ 2m = 14), KMP = 18 (≤ 2n = 30), naif = 24 · kart: hep Θ(n + m), ek bellek O(m)

Hata Avcısı: altı hata, bu makinede gözlenen belirtiler

#	Hata	Belirti (gerçek çıktı)	Çözüm
1	Negatif karma, % ile negatif indis (C)	-57261580, indis = h % 11 = -2 şule → munmap_chunk(): invalid pointer (SIGABRT)	uint32_t karma, unsigned char bayt
2	Anahtar dizgisi kopyalanmıyor (C)	bütün kovalarda deniz; ara ali: YOK	Düğüm kendi kopyasını tutar (malloc + memcpy)
3	Silinen hücre BOŞ yapılıyor	ara ece: YOK — tabloda olduğu hâlde! (3 yoklama)	Mezar taşı: arama durmaz
4	Yük denetimi yok (C++)	ekle ılgın: 1000000 yoklamadan sonra DURDURULDU — BOŞ hücre yok	(n + 1)/m > 3/4 ise × 2
5	Rabin–Karp'ta doğrulama yok	bildirilen eşleşme: 7578 (doğrusu 1)	Adayı memcmp ile doğrula
6	Python'da list anahtarı	TypeError: unhashable type: 'list'	tuple(sorted(s)) ya da bytes

Hata Avcısı 5: karma eşitliği eşleşme değildir

hata_avcisi/H10_HA5_dogrulamasiz_rabin_karp.py · satır 31–34 (hatalı) — Python

```python
        if hp == ht:                        # HATA: doğrulama yok
            if say < 5:
                konum.append(s)
            say += 1
```

hata_avcisi/H10_HA5_dogrulamasiz_rabin_karp_duzeltilmis.py · satır 31–37 — Python

```python
        if hp == ht:
            if t[s:s + m] == p:             # DÜZELTME: doğrula
                if say < 5:
                    konum.append(s)
                say += 1
            else:
                sahte += 1
```

HATALI: BELİRTİ

```
konumlar: 6 12
bildirilen eşleşme: 2
bildirilen eşleşme: 7578  (rastgele metin)
```

DÜZELTİLMİŞ

```
konumlar: 6; sahte eşleşme (elenen) = 1
konumlar: 50000; sahte eşleşme (elenen) = 7577
```

Dil Penceresi: karma tabloları üç dilde

	C	C++	Python
Genel tablo	standartta yok (POSIX hsearch sınırlı)	unordered_map / unordered_set	dict / set
Yük ve büyüme	programcı (Örnek 6)	max_load_factor() = 1,0; rehash, reserve	yorumlayıcı (CPython 2/3)
Dolaşma sırası	programcının yapısı	belirtilmemiş	dict: ekleme sırası (3.7+); set: güvence yok
Dizgi karması	elle (FNV-1a, polinom)	std::hash: gerçekleştirime bağlı	SipHash, süreç başına tohum (PYTHONHASHSEED)
Kullanıcı türü anahtar	karma + eşitlik elle	operator== + std::hash<T>	__eq__ + __hash__ (değişmez)
Küçük harf	tolower: bayt bayt, yerel ayar	std::tolower (aynı)	str.lower: Unicode (İ → i)
-7 % 2	−1 (bölünenin işareti)	−1	1 (bölenin işareti)

Mühendis Gözüyle · Atölyeler · İleri Bakış · Merak

Mühendis Gözüyle

- “O(1)” bir varsayımdır: dışarıdan gelen anahtara anahtarlı karma + girdi sınırı.
- Yük eşiği bir bellek–zaman ödünleşimi: başarısız arama $\alpha = 0{,}75$'te 8,5, 0,95'te 200,5 yoklama.
- Sıralı dolaşma, aralık sorgusu gerekiyorsa karma tablosu değil.

Atölyeler

- Karma fonksiyonu yarışması: 10^5 ad, 100 003 kova, χ^2.
- Kelime sayacı: üç dil, elle yazılmış tablo ↔ kütüphane.
- Saldırı ve savunma: çakışan bloklar, dört savunma.

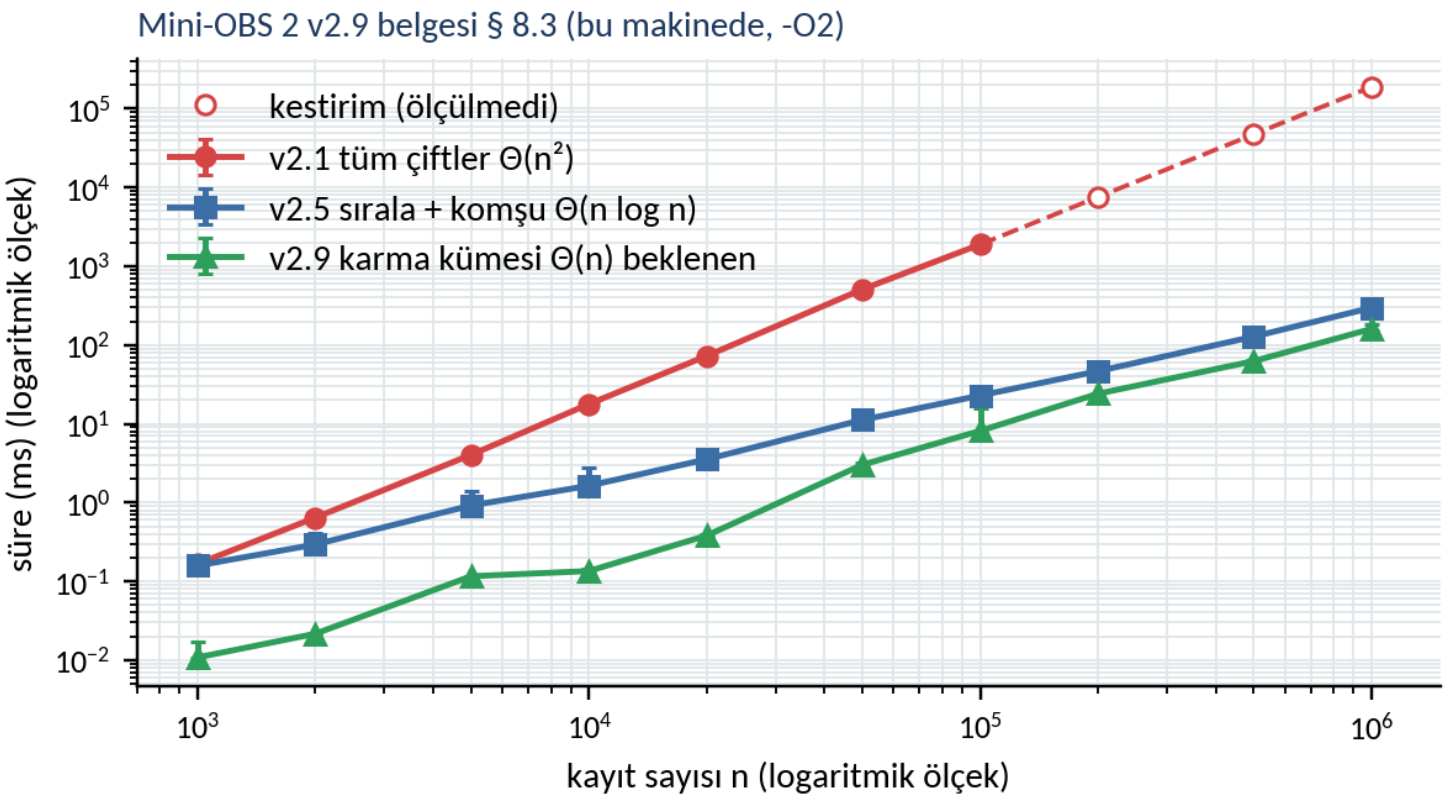
İleri Bakış

- H11 ziyaret kümesi; H13 memoization; SHA-2/SHA-3 kriptografide.
- Bloom filtresi, MinHash, tutarlı karma; sonek dizileri, Aho–Corasick.
- **Hazırlık (H11):** Bölüm 4 ve 6, karma tablosu oynatıcısı, Alıştırma 10.1–10.8.

Merak

- KMP'nin π'si en kısa periyodu da verir: $m - \pi[m]$ (ababaca: 6).
- P$T dizgisinin önek fonksiyonu neden bütün eşleşmeleri verir?

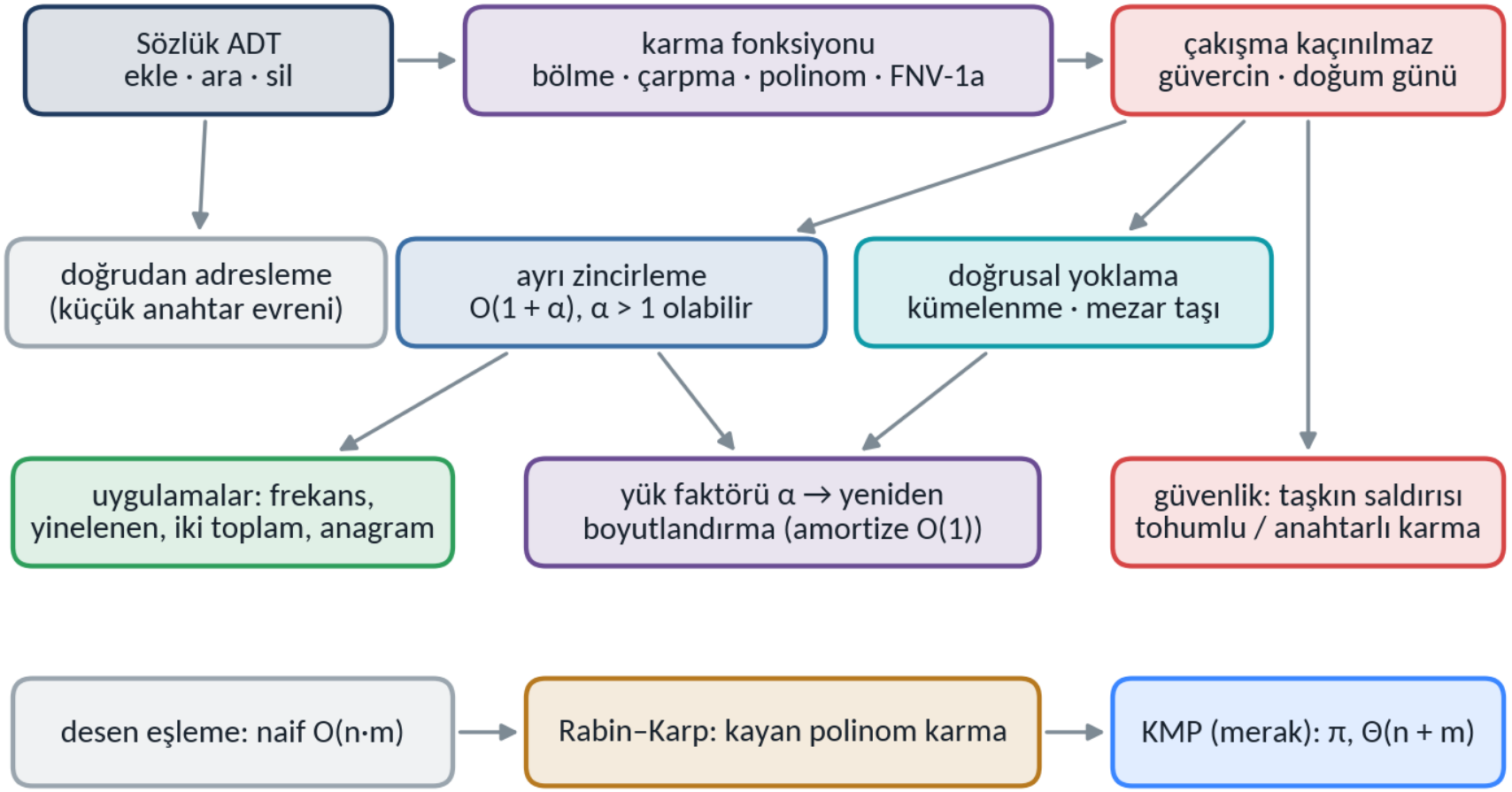
Mini-OBS 2 v2.9: numara dizini ve O(n) yinelenen tespiti



- **Numara → kayıt karma tablosu:** açık adresleme, doğrusal yoklama, mezar taşı; (n + mezar)/m > 0,7 olacaksa × 2.
- **Yinelenen tespiti** karma kümesiyle beklenen Θ(n): v2.1 (H2) ve v2.5 (H6) ile aynı veride, aynı çiftler.
- **Alt dizgi araması** ad–soyadda büyük/küçük harf duyarsız: naif ve Rabin–Karp.
- **Kabul ölçütleri:** 8 denetlenebilir madde (v2.9 belgesi § 10).

v2.9 belgesi § 8.3, n = 10⁶: tüm çiftler ≈ 188,96 s (kestirim), sırala + komşu 297,2 ms, karma kümesi 160,2 ms

Özet: karma tablosu ve desen eşleme



Karma → kova → çakışma çözümü (zincirleme / yoklama + mezar taşı) → yük ve büyütme → uygulamalar, güvenlik, kayan karma

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. Boş tabloda m = 7, h(k) = k mod 7, doğrusal yoklama: 10, 17, 24 sırayla eklenince 24 hangi hücreye, kaç yoklamayla yerleşir?

2. Rabin–Karp karma eşitliğinde pencereyi neden yine karakter karakter karşılaştırır? q = 13 yerine q = 1 000 003 seçilirse ne değişir?

3. Python'da `{[1, 2]: "a"}` neden hata verir? Anahtarı nasıl düzeltirsiniz?